

```

import { q, one, j } from '../db.js';
import { subscribe } from './events.js';

// An automation is data, not code: a trigger and a list of capability calls.
// One business writes it, publishes it, and any other business can install it.
// Publishing a new version updates every business that installed it, because
// they hold a reference to the key, not a copy of the definition.
//
// { on: "forms.submitted", do: [ { capability: "crm.upsert_contact", input: {

export async function publish({ businessId, key, version, name, definition, visib
  validate(definition);
  return one(
    `insert into shared_automation (author_business_id, key, version, name, defin
    values ($1,$2,$3,$4,$5::jsonb,$6)
    on conflict (key, version) do update set definition = excluded.definition,
      name = excluded.name, visibility = excluded.visibility
    returning *`,
    [businessId, key, version, name, JSON.stringify(definition), visibility]
  );
}

export async function latest(key) {
  return one(
    `select * from shared_automation where key = $1 and visibility = 'marketplace
    order by created_at desc limit 1`, [key]
  );
}

export async function marketplace() {
  return q(
    `select key, name, version, author_business_id, created_at from shared_automa
    where visibility = 'marketplace' order by created_at desc`
  );
}

// Installing records the key. It does not copy the definition.
export async function installAutomation({ businessId, key }) {
  const found = await latest(key);
  if (!found) throw new Error(`no published automation "${key}"`);
  return one(
    `insert into install (business_id, package_id, package_key, version, settings
    values ($1, null, $2, $3, '{}::jsonb, 'active')
    on conflict (business_id, package_key) do update set status = 'active', vers

```

```

        returning *`,
        [businessId, `automation:${key}`, found.version]
    );
}

function validate(def) {
    if (!def.on) throw new Error('automation needs a trigger topic');
    if (!Array.isArray(def.do) || def.do.length === 0) throw new Error('automation
    for (const step of def.do) {
        if (!step.capability) throw new Error('every step needs a capability');
    }
}

// Resolve {{payload.x}} against the triggering event.
function bind(value, payload) {
    if (typeof value === 'string') {
        return value.replace(/\{\{payload\.([a-z0-9_\.]+\)\}\}/gi, (_, path) => {
            const v = path.split('.').reduce((o, k) => o?.[k], payload);
            return v === undefined ? '' : String(v);
        });
    }
    if (Array.isArray(value)) return value.map(v => bind(v, payload));
    if (value && typeof value === 'object') {
        return Object.fromEntries(Object.entries(value).map(([k, v]) => [k, bind(v, p
    }
    return value;
}

const topics = new Set();

// Wire a topic once; every business that installed an automation on it runs.
export async function watch(topic) {
    if (topics.has(topic)) return;
    topics.add(topic);

    subscribe(topic, async ({ businessId, payload }) => {
        if (!businessId) return;
        const installs = await q(
            `select package_key, version from install
            where business_id = $1 and status = 'active' and package_key like 'automa
            [businessId]
        );
        if (installs.length === 0) return;

        const { request } = await import('./executor.js');
        const business = await one(`select * from business where id = $1`, [businessI
        const ctx = { businessId, business, person: null, membership: { id: null, rol

```

```
for (const inst of installs) {
  const key = inst.package_key.slice('automation:'.length);
  const found = await latest(key);
  if (!found) continue;
  const def = j(found.definition);
  if (def.on !== topic) continue;

  for (const step of def.do) {
    await request({ ctx, capability: step.capability, input: bind(step.input
  }
}
});
}
```